Review of C++ Fundamentals

Appendix A

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Introduction to C++

- C++ is an object-oriented language based on C
- C++ was developed by Bjorne Stroustrap at Bell Labs (1980's)
- CSC 340 doesn't assume experience in C or C++
- CSC 340 does assume experience with Java

C++ Syntax

```
1. Enables input and output ------------>   #include <iostream>
2. Opens the standard namespace ---->       using namespace std;   // syntax structure allowing grouping of data & methods
3. Begins the function main ------------>   int main()             // main must be global and return int
4. A comment ---------------------------->  // Computes the volume of a sphere of a given radius.
5. Begins body of function -------------->  {
6. Defines a constant ------------------->     const double PI = 3.14159;   // constants do not change
7. Declares a variable ------------------->    double radius;   // variables begin with a letter or underscore

8. Displays a prompt to the user ------->      cout << "Enter the radius of the sphere: ";
9. Reads radius ------------------------->    cin  >> radius;

10. Declares and computes volume --->          double volume = 4 * PI * radius * radius * radius / 3;
11. Displays results ------------------------>  cout << "The volume of a sphere of radius "
12. Statement continues ----------------->          << radius << " inches is " << volume
13. Statement continues ---------------->           << " cubic inches.\n";
14. Normal program termination ------->        return 0;
15. Ends body of function --------------->  }  // end program
```

Multi-line comments start with /* and end with */

C++ Functions

- An argument passed by value is copied within the function.
- The function cannot alter the arguments passed in.

```
int computeMax(int x, int y)
{
   if (x > y)
      return x;
   else
      return y;
}  // end computeMax
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Functions

- An argument passed by reference is not copied but is accessed directly within the function.

```
void computeMax(int x, int y, int& larger)        // void function does not return a value
{
        larger = ((x > y) ? x : y):
}       // end computeMax

int main ()
{
         int a = 0;
         int b = 0;
         int largerAB = 0;
         cin >> a >> b;
         computeMax(a, b, largerAB);
         cout << "The larger of " << a << " and " << b << " is " << largerAB << ".\n";
}
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Functions

- A constant reference parameter is a reference parameter that is tagged as const.
- The function uses the actual argument that is passed to such a parameter, not a copy of it, yet cannot modify it.
- For example, for the function f that begins

```
void f( const int& x, int y, int& z)
```

x is a constant reference parameter

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Passing Arrays to a Functions

- Arrays are always passed by reference to a function.
- Common ways to pass arrays to function:
  1. void modifyArray(int arr[]) // same as #2
  2. void modifyArray(int *arr) // same as #1
  3. void modifyPointer(int*& arr) // allows you modify array elements and pointer itself; useful when you want to change memory location

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

By Reference or By Value?

- Parameters are passed by value
- Copy of the parameter is made on stack
- If the parameter is a class, it's by value (class is copied)
- If the parameter is a pointer/address, pointer itself is still by value (i.e., pointer is copied)

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Vectors

- A vector is similar to a one-dimensional array.
  - an object of a standard C++ class named vector
  - is a part of the Standard Template Library , or STL
    - The STL is a collection of data types you can use in your programs.
    - These data types are not part of the official C++ language, but they have been added to the built-in data types.
- To use a vector in your program, you must begin it with the following statements:

```
#include <vector>
using namespace std;
```

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Vectors

- You can declare a vector in one of three ways:
  1. Place the type of data it will hold in angle brackets and the number of elements in parentheses:
     a) vector<double> firstVector(10); // Vector to hold 10 doubles
     b) vector <string> myVector(12); // Vector to hold 12 strings
     c) The size you specify when declaring the vector is only its initial size. A vector can grow in size when you add entries.
  2. You can place initial values into a vector when you declare it, by writing a second argument:
     a) vector<int> intVector(5, -1); // Vector to hold 5 integers, initialized to -1
     b) When the elements in the vector are allocated, they are given the value of the second argument.
  3. You can also create an empty vector—a vector with no elements—by omitting its size and the parentheses:
     a) vector<char> letterVector; // An empty vector of characters

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Vectors

- Using [] stores a value in an existing element, the same way when using an array.

```
double x = firstVector[5];  // Places sixth entry in x
myVector[3] = "This is a sample string."; // Sets fourth entry's value
```

- If you are not sure how many elements the vector has, you can call the method size, as in the following example:

```
cout << intVector.size() << endl; // Displays the number of elements in intVector
```

- If vector is full, we could use the push_ back method to add additional values:

```
firstVector.push_back(2.3); // Grow vector and store value 2.3
firstVector.push_back(3.4); // Grow vector and store value 3.4
```

- At this point, calling the size method would be updated, since two additional elements have been added to the vector.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Vectors

- You also can reduce the size of a vector by removing either its last element or all of its elements.
- To remove only the last element, you can use the pop_back method.
- This method shortens the vector but does not give you the entry in the removed element.
- You must save that entry before calling pop_back.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Strings

- The C++ Standard Library provides the data type string. To use this library, you include the statements

```
#include <string>
using namespace std;
```

- You can concatenate two strings to form another string by using the + operator.
- You can manipulate a portion of a string by using the method

```
substr(position, length )
```

- The first argument specifies the position of the beginning of the substring (remember that 0 is the position of the first character in the string). The second argument is the length of the substring. For example, (e.g., title = "J Perfect's Diary");

```
title.substr(2, 7)
```

- is the string “ Perfect ”.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Classes

- In C++, a class has the following form:

```
class Sphere
{
private:
    // Declarations of private data members and methods
    // Definitions of private methods
    . . .

public:
    // Definitions of public methods
    . . .
};
```

- By default, all members in a class are private —they are not directly accessible by any program that uses the class—unless you designate them as public. However, explicitly indicating the private and public portions of a class is a good programming practice.
- An object is an instance of a class.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Constructors and Destructors

- Classes have special methods, called constructors and destructors, for the creation and destruction of its objects.
- A constructor creates and initializes new objects, or instances, of a class.
- A destructor destroys an instance of a class, when the object's lifetime ends.
- A typical class has several constructors, but only one destructor. For many classes, you can omit the destructor. In such cases, the compiler will generate a destructor for you.
- In C++, a constructor has the same name as the class. Constructors have no return type—not even void —and cannot use return to return a value. Constructors can have arguments.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Constructors and Destructors

- The first constructor in Sphere is the default constructor :

  Sphere();

- A default constructor by definition has no arguments. Typically, a default constructor initializes data members to values that the class implementation chooses. For example, the implementation

```
Sphere::Sphere()
{
    theRadius = 1.0;
}   // end default constructor
```

- sets theRadius to 1.0.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Constructors and Destructors

- When you declare an instance of the class, you implicitly invoke a constructor. For example, the statement

```
Sphere unitSphere;
```

- invokes the default constructor, which creates the object unitSphere and sets its radius to 1.0. Notice that you do not include parentheses after unitSphere.
- The next constructor in Sphere is

```
Sphere(double initialRadius);
```

- It creates a sphere object of radius initialRadius. This constructor needs only to initialize the private data member theRadius to initialRadius. Its implementation is

```
Sphere::Sphere(double initialRadius)
{
   theRadius = initialRadius;
} // end constructor
```

- You implicitly invoke this constructor by writing a declaration such as

```
Sphere mySphere(5.1);
```

- Typically, you place the implementation of a class's constructors and other methods in an implementation file whose name ends in .cpp.

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

End

Appendix A

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

Appendix

Appendix A

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Basics

Key Points

1.C++ source files end with the **.cpp** extension
2.#include is a **preprocessor directive**. Preprocessor directives begin with the symbol #.
3.The **cout** object along with the stream insertion operator (**<<**) can be used to display a string on the console.
4.Every C++ program is executed from a **main** function. A function is a construct that contains statements.
5.Every **statement** in C++ must end with a semicolon (**;**), known as construct that contains statements.
6.In C++, a comment is preceded by 2 forward slashed (**//**) on a line, called a **line comment**, or enclosed between **/*** and ***/** on one or several lines, called a **block comment** or **paragraph comment**.
7.Keywords, or reserved words, have a specific meaning to the compiler and cannot be used in the program for other purposes. Examples of **keywords** are **using**, **namespace**, **int**, and **return**.

8.C++ source programs are **case sensitive**.
9.You can develop C++ applications from the command window or by using an IDE such as Visual Studio or Xcode.
10.Programming errors can be categorized into 3 types: syntax errors, runtime errors, and

C++ Basics

Key Points

1.The cin object along with the stream extraction operator (>>) can be used to read an input from the console.
2.Identifiers are names for naming elements in a program. An identifier is a sequence of characters that consists of letters, digits, and underscores (_). An identifier must start with a letter or an underscore. It cannot start with a digit. An identifier cannot be a reserved word.
3.Choosing descriptive identifiers can make programs easy to read.
4.Declaring a variable tells the compiler what type of data a variable can hold.
5.In C++, the equal sign (=) is used as the assignment operator.
6.A variable declared in a function must be assigned a value. Otherwise, the variable is called uninitialized and its value is unpredictable.
7.A named constant or simply constant represents permanent data that never changes.
8.A named constant is declared by using the keyword const.
9.By convention, constants are named in uppercase.
10.C++ provides integer types (short, int, long, unsigned, short, unsigned int, and unsigned long) that represent signed and unsigned integers of various sizes.
11.Unsigned integers are nonnegative integers.
12.C++ provides floating-points types (float, double, and long double) that represent

C++ Basics

Key Points

13. C++ provides operators that perform numeric operations: + (addition), - (subtraction), * (multiplication), / (division), and % (modulus).
14. Integer arithmetic (/) yields an integer result.
15. In C++, the % operator is for integers only.
16. The numeric operators in a C++ expression are applied the same way as in an arithmetic expression.
17. The increment operator (++) and the decrement operator (--) increment or decrement a variable by 1.
18. C++ provides augmented operators += (addition assignment), -= (subtraction assignment), *= (multiplication assignment), /= (division assignment), and %= (modulus assignment).
19. When evaluating an expression with values of mixed types, C++ automatically casts the operands to appropriate types.
20. You can explicitly cast a value from one type to the other using the <static_cast>(type) notation or the legacy c-style (type) notation.
21. In computer science, midnight of January 1, 1970 is known as the UNIX epoch.

C++ Basics

Key Points

1.A bool type variable can store a true or false value.
2.Internally, C++ uses 1 to represent true and 0 for false.
3.If you display a bool value to the console, 1 is displayed I the value is true and 0 if the value is false.
4.In C++ you can assign a numeric value to a bool variable. Any nonzero value evaluates to true and zero value evaluates to false.
5.The relational operators (<, <=, ==, !=, >, >=) yield a Boolen value.
6.The quality testing operator is two equal signs (==), not a single equal sign (=). The later symbol is for assignment.
7.Selection statements are used for programming with alternative courses of actions. There are several types of selection statements: if statements, two-way if-else statements, nested if statement, multi-way if-else statements, switch statements, and conditional expressions.
8.The various if statements all make control decisions based on a Boolean expression. Based on the true and false evaluation of the expression, these statements take one of two possible courses.
9.The Boolean operators &&, ||, and ! Operate with Boolean values and variables.

C++ Basics

Key Points

10. When evaluating p1 && p2, C++ first evaluates p1 and then evaluates p2 if p1 is true; if p1 is false, it does not evaluate p2.
11. When evaluating p1 || p2, C++ first evaluates p1 and then evaluates p2 if p1 is false; if p1 is true, it does not evaluate p2.
12. Therefore, && is referred to as the conditional or short-circuit AND operator, and || is referred to as the conditional or short-circuit OR operator.
13. The switch statement makes control decisions based on a switch expression.
14. They keyword break is optional in switch statement, but it is normally used at the end of each case in order to skip the remainder of the switch statement. If the break statement is not present, the next case statement will be executed.
15. The operators in expressions are evaluated in the order determined by the rules of parentheses, operator precedence, and operator associativity.
16. Parentheses can be used to force the order of evaluation to occur in any sequence.
17. Operators with higher precedence are evaluated earlier. For operators of the same precedence, their associativity determines the order of evaluation.
18. All binary operators except assignment operators are left-associative; assignment operators are right-associative.

C++ Basics

Key Points

1.C++ provides the mathematical functions sin, cos, tan, asin, acos, atan, exp, log, log10, pow, sqrt, cell, floor, min, max, and abs for performing mathematical functions.
2.Character type (char) represents a single character.
3.The character \ is an escape character and an escape sequence starts with the escape
character followed by another character or a combination of digits.
4.C++ allows you to use escape sequences to represent special characters such as '\t' and '\n'.
5.The characters ' ', '\t', '\f', '\r', and '\n' are known as the whitespace characters.
6.C++ provides the functions isdigit, isalpha, isalnum, islower, isupper, isspace for testing whether a character is a digit, letter, digit or letter, lowercase, uppercase, and whitespace. It also contains the tolower and to toupper functions for returning a lowercase or uppercase letter.
7.A string is a sequence of characters. A string value is enclosed in matching double quotes(“). A character value is enclosed in matching single quotes “’”.
8.You can declare a string object using the string type. A function that is invoked from a specific object is called an instance function.
9.You can get the length of a string by invoking its length() function, and retrieve a character at the specified index in the string using the at(index)

C++ Basics

Key Points

11. You can use the relational operators to compare two strings.
12. You can format output using stream manipulators defined in the iomanip header.
13. You can create an ifstream object for reading data from a file and an ofstream object for writing data to a file.

C++ Basics

Variables:

•A variable, whose name is a C++ identifier, represents **a memory location** that contains a value of a particular data type.

•When possible, **avoid uninitialized variables of a primitive data type**.

Literal Constants vs. Named Constants:

- int i = 4; // 4 is a literal constant
- **const** double PI = 3.14159; // PI is a named constant, a named floating constant
  // same as **final** in Java

Enumerations:

•enum {SUN, MON, TUE, WED, THU, FRI, SAT}; // const int SUN = 0; and so on

•By default, the values assigned to the constants --- called enumerators--- begin with zero and are consecutive.

•To assign explicit value: enum {PLUS = '+', MINUS = '-'};

•Naming an enumeration makes a distinct integral type:

- enum Season {WINTER, SPRING, SUMMER, FALL};
- Season whichSeason; // whichSeason now has all the 4 seasons.

The typedef Statement:

•Does not create a new type but a new name/alias/shortcut for an existing data type

- typedef double Real; // Can use 'Real' for a group of variables. Easy to manage.

C++ Basics

C++ Data Types:

1.Primitive Data Types: These data types are built-in or predefined data types and can be used directly by the user to declare variables: int, char , float, bool…

2.Abstract or user defined data type: These data types are defined by user like defining a class in C++ or a structure.

Primitive Types:

- Java has 8 primitive types: boolean, byte, char, short, int, long, float, and double
- Java primitives have precise sets of possible values on
- C++ primitive types have unspecified ranges.

Integer Types:

- **int** can be 16, 32, or 64 bits, depending on the platform.
- The precise range also vary from machine to machine.
- int can be augmented with **short** or **long**.
- short and long can be by themselves without int.
- 2 modifiers for int are **signed** and **unsigned**. Also applied to any of the integer types (or char).
- unsigned int are never negative
- **short** ranges -32, 768 to 32,767 and **unsigned short** ranges 0 to 65,535 because unsigned instructs the compiler to interpret what would normally be the sign bit as an extra bit.

C++ Basics

Floating Point Types:

- C++ floating point types include: float and double
- Like Java, a double provides more significant digits than a float

Char Type:

- Stores characters but unspecified size and unspecified signed or unsigned.
- Each Java character is 16 bit Unicode.
- C++ compilers usually use 8 bits and store ASCII characters.
- C++ has all the Java escape sequences and
  - \a for the bell, \? for question mark, and \v for vertical tab
- Include **cctype** to use isupper, islower, isalpha, toupper and tolower… like java.lang.Character
- C++ now supports wcahr_t type and wide-character literal

Boolean Type:

- bool has values: true or false. But they are actual integers 0 and 1.
- 0 means false.
- Nonzero means true.
- A null pointer is false. Otherwise, true.

C++ Basics

string Type

```
#include <string>
string message = ”Welcome to C++";
cout << message << endl;
```

- C++ strings are easier to use than Java strings
- The equality and relational operators ==, !=, <, <=, >, => work for C++ strings.
- When = is applied to a string, a copy is made. Changes to the original do not affect the copy.
- Java strings are **immutable**.
- C++ strings are **mutable**.
  - Using the array indexing operator [] can access and **change individual characters**.
  - += does the job for Java StringBuilder
- C++ substr()
  - C++ parameters: starting position and length of substring
  - Java parameters: starting position and first nonincluded position
- To obtain a equivalent primitive strings, instead of the string library type, use c_str string member function.

C++ Basics

string Type

•**Compare strings:** use the **relational operators** ==, !=, <, <=, >, >= to compare two strings. This is done by comparing their **corresponding characters** on by one from **left to right**.

```
string s1 =      // ASCII C
"ABC"; string    67
s2 = "ABE";      // ASCII E
                 69
cout << (s1 == s2)   // Displays 0
<< endl; cout << (s1   (means false)
!= s2) << endl; cout   // Displays 1
cout << (s1 >= s2)   (means true)
<< endl; cout << (s1   // Displays 0
> s2) << endl;        (means false)
cout << (s1 <= s2)   // Displays 0
<< endl;             (means false)
                     // Displays 1
                     (means true)
                     // Displays 1
                     (means true)
```

C++ Basics

string Type

- **Read strings:**

```
string city;
cout << "Enter a city: ";        // Read to string city
cin >> city;
getline(cin, city, '\n');        // Same as getline(cin, city)
```

function

std::getline (string) `<string>`

C++98 | C++11

```
(1) istream& getline (istream&  is, string& str, char delim);
    istream& getline (istream&& is, string& str, char delim);
(2) istream& getline (istream&  is, string& str);
    istream& getline (istream&& is, string& str);
```

Get line from stream into string

Extracts characters from *is* and stores them into *str* until the delimitation character *delim* is found (or the newline character, `'\n'`, for *(2)*).

The extraction also stops if the end of file is reached in *is* or if some other error occurs during the input operation.

If the delimiter is found, it is extracted and discarded (i.e. it is not stored and the next input operation will begin after it).

Note that any content in *str* before the call is replaced by the newly extracted sequence.

Each extracted character is appended to the string as if its member push_back was called.

Source: http://www.cplusplus.com/reference/string/string/getline/

C++ Basics

string Type

- **Simple File Output:**

- To write data to a file, first declare a variable of the ofstream type:
  - ofstream output;

- To specify a file, invoke the open function from output object as follows:
  - output.open("numbers.txt");

- Optionally, you can create a file output object and open the file in one statement like this:
  - ofstream output("numbers.txt");

- To write data, use the stream insertion operator (<<) in the same way that you send data to the cout object:
  - output << 95 << " " << 56 << " " << 34 << endl;

C++ Basics

string Type

- **Simple File Input:**

- To read data from a file, first declare a variable of the ifstream type:
  - ifstream input;

- To specify a file, invoke the open function from input as follows:
  - input.open("numbers.txt");

- Optionally, you can create a file output object and open the file in one statement like this:
  - ifstream input("numbers.txt");

- To read data, use the stream extraction operator (>>) in the same way that you read data from the cin object:
  - input << score1 << score2 << score3;

C++ Review

- Enumerations provides another way to name integer constants
  - enum {SUN, MON, TUE, WED, THU, FRI, SAT};
- Is equivalent to:
  - const int SUN = 0;
  - const int MON = 1;
  - . . .
  - const int SAT = 6;
- easier to read

© 2017 Pearson Education, Hoboken, NJ. All rights reserved

C++ Basics

Implicit type conversions:

•Automatic conversion from one data type to another can occur during assignment and during expression evaluation.

•For assignments, the data type of the expression on the right-hand side of the assignment operator is converted to the data type of the item on the left- hand side just before the assignment occurs. Floating-point values are truncated, not rounded, when they are converted integral values.

•**Integral promotion:** If the operands of an operator differ in data type, the data type that is lower in the following hierarchy is converted to on that is higher (int is lowest):
int ➔ unsigned ➔ long ➔ unsigned long ➔ float ➔ double ➔ long double

•**Explicit type conversions**:

- To explicitly convert from one data type to another, use **static cast**:

C++ Basics

Primitive Built-in Types:

•C++ offers the programmer a rich assortment of built-in as well as user defined data types. Following table lists down seven basic C++ data types:

•Several of the basic types can be modified using one or more of these type modifiers:

- signed
- unsigned
- short
- long

•The table, next slide, shows the variable type, how much memory it takes to store the value in memory, and what is maximum and minimum value which can be stored in such type of variables.

•The size of variables might be different from those shown in the above table, depending on the compiler and the computer you are using.

Category	Data Types
Boolean	bool
Character	char | signed char | unsigned char
Signed Integer	short |int | long
Unsigned Integer	unsigned short | unsigned |unsigned long
Floating point	float | double | long double
Valueless	void
Wide character	wchar_t

•Use the **sizeof** function to find the size of a type on a specific machine:

```
cout << sizeof(int) << " " << sizeof(long) << " " <<
sizeof(double);
```

C++ Basics

Conditionals

- **If** statement is C++ is identical to the Java's except that in C++ the condition of the if statement can be either **int** or a **bool**.
- If (x = 4)
  - This is not a complier error.
  - x is assigned/reassigned to be 4
  - The condition is evaluated **true**
  - **bool**: 0 means false and nonzero means true

- **Conditional expression**:
  - larger = ((a > b) ? a : b); // Conditional operator. Ternary operator.
  - cout << ((a > b) ? "a is greater" : "a is not greater");

- **Switch** statement is identical to the Java's.

C++ Basics

Type	Typical Bit Width	Typical Range
char	1 byte	-127 to 127 or 0 to 255
unsigned char	1 byte	0 to 255
signed char	1 byte	-127 to 127
int	4 bytes	-2147483648 to 2147483647
unsigned int	4 bytes	0 to 4294967295
signed int	4 bytes	-2147483648 to 2147483647
short int	2 bytes	-32768 to 32767
unsigned short int	Range	0 to 65,535
signed short int	Range	-32768 to 32767
long int	4 bytes	-2,147,483,648 to 2,147,483,647
signed long int	4 bytes	same as long int
unsigned long int	4 bytes	0 to 4,294,967,295
float	4 bytes	+/- 3.4e +/- 38 (~7 digits)
double	8 bytes	+/- 1.7e +/- 308 (~15 digits)
long double	8 bytes	+/- 1.7e +/- 308 (~15 digits)
wchar_t	2 or 4 bytes	1 wide character

Source: https://www.tutorialspoint.com/cplusplus/cpp_data_types.htm

C++ Basics

cmath header

Trigonometric

Function	Description
`sin(radians)`	Returns the trigonometric sine of an angle in radians. Returns the trigonometric cosine of an angle in radians. Returns the trigonometric tangent of an angle in radians. Returns the angle in radians for the inverse of sine.
`cos(radians)`	
`tan(radians)`	
`asin(a)`	
`acos(a)`	Returns the angle in radians for the inverse of cosine. Returns the angle in radians for the inverse of tangent.
`atan(a)`	

max(**2**, **3**) returns **3**

max(**2.5**, **3.0**) returns **4.0**

min(**2.5**, **4.6**) returns **2.5**

abs(**-2**) returns **2**

abs(**-2.1**) returns **2.1**

cmath header

C++ BASICS

Exponent

Function	Description
exp(x)	Returns e raised to power of x (e^x).
log(x)	Returns the natural logarithm of x (ln(x) = $\log_e(x)$).
log10(x)	Returns the base 10 logarithm of x ($\log_{10}(x)$).
pow(a, b)	Returns a raised to the power of b (a^b).
sqrt(x)	Returns the square root of x ($\sqrt{x}$) for x >= 0.

Rounding

Function	Description
ceil(x)	x is rounded up to its nearest integer. This integer is returned as a double value.
floor(x)	x is rounded down to its nearest integer. This integer is returned as a double value.

Unsigned and Negative Integers

- **Unsigned integers** can only hold non-negative numbers: unsigned u = 0;
- Assigning a **negative value** to an unsigned integer can lead to confusion:
  - unsigned int u = -4;
  - count << u << endl; // 4294967292
- Integer division involving **negative numbers** is platform dependent: -23/4 = -5 or -6
- Right shifting a negative number is machine dependent.
- Negative numbers should not be used with remainder because it is machine dependent: 21 % (-5) = - 1 or 1
- Math routines use double, not float ➔ **use double**

C++ Memory Management

- In Java, all objects are created through `new`, and the memory will be reclaimed with the garbage collector when no longer referenced
- In C++, objects can be created via `new`, but the memory is not automatically reclaimed
- In C++, if an object doesn't have a pointer to it, it is a memory leak because it can't be reclaimed.
- Memory leaks are very bad and eventually will bring the software down

C++ and Standard Template Library (STL)

- Functionally, similar to Java generics
- In C++, best to explicitly include only what is needed

```
#include <list>
```

- In C, code is in .c files and interfaces/declarations in .h (header files)
  - In C++, header files can contain implementation
  - .hpp is a convention to differentiate from C headers
- However, template code is contained in .hpp files
  - injected at compile time